TRƯỜNG ĐẠI HỌC FPT

LAB 1 REPORT

SPACE EXPLORER — 2D ARCADE GAME DEVELOPMENT

STUDENT NAME	NGUYỄN CÔNG NHẤT
STUDENT ID	QE190017
CLASS/ COURSE	SE19B.NET — PRU213

Table of Contents

1. Project Overview

1.1. Introduction

Space Explorer is a 2D arcade space-shooter game developed using the Unity Engine (version 6, C#). The project demonstrates the application of fundamental game development principles including object-oriented programming, physics-based collision detection, dynamic object spawning, score-driven difficulty scaling, and a multi-scene UI flow.

The game features a continuously escalating difficulty system where the player must survive an asteroid field, collect energy stars, and periodically fight a Boss enemy that appears every 5 levels.

1.2. Game Objective

The player pilots a spaceship through an increasingly dangerous asteroid field. Core objectives:

· Survive — any collision with a falling asteroid or a boss projectile results in immediate Game Over.

· Score Points — destroy asteroids with laser fire (+5 pts each) or collect floating Energy Stars (+10 pts each).

· Defeat the Boss — a Boss enemy with 30 HP spawns every 5 levels.

· Reach Higher Levels — every 100 points advances the player one level, increasing asteroid speed, spawn density, and background depth.

2. Pre-Lab Setup and Environment Configuration

2.1. Display and Camera Configuration

The Main Camera was configured with an Orthographic projection to guarantee a flat, 2D rendering plane with no perspective distortion. The game resolution is locked to 16:9 aspect ratio, ensuring consistent boundary calculations for all spawn points (x: -8f to +8f, spawn row at y: 6f).

The Application.targetFrameRate = 60 and QualitySettings.vSyncCount = 0 flags were set programmatically in GameManager.Start() to enforce a stable 60 FPS cap across different hardware profiles.

2.2. Tag System Initialization

A structured Tag system was declared in Unity's Tag Manager and assigned to their respective Prefabs to power the collision detection matrix:

Tag	Assigned To	Purpose
Player	Player.prefab (root)	Asteroids and boss bullets detect the player
Laser	LaserBullet 1.prefab	Boss and asteroids detect incoming fire

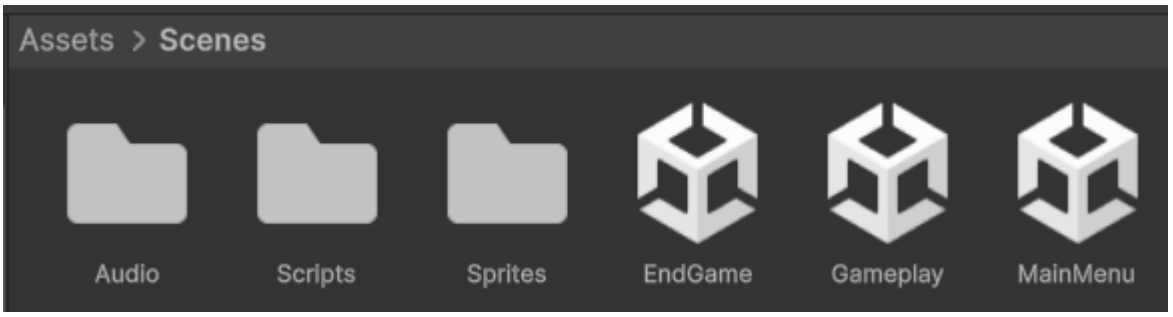
Enemy	Asteroid.prefab	Reserved for future use
Star	Star.prefab	Player collects stars via trigger

2.3. Project Directory Organization

The Assets/ folder was structured according to Unity best practices:

```
Assets/
├── Prefabs/      → All reusable GameObjects (Asteroid, Boss, Player, Star,
EnemyBullet, PowerUps…)
├── Scenes/
│   ├── Audio/    → BGM and SFX .wav/.mp3 clips
│   ├── Backgrounds/ → Background sprite sets (Far/Mid/Near layers)
│   ├── Scripts/  → All C# MonoBehaviour controllers
│   └── Sprites/  → Asteroid, Boss, Ship, Star, PowerUp sprites
└── _Recovery/    → Backup copies of critical scripts
```

IMAGE 1: Screenshot of the organized Assets folder structure in the Unity Editor Project window.



3. Scene Management & User Interface (UI)

3.1. Build Settings and Scene Flow

The project is composed of 3 scenes registered in Build Settings in the following index order:

Index	Scene Name	Role
0	MainMenu	Title screen, navigation hub

1	Gameplay	Core game loop
2	EndGame	Score display and replay prompt

Scene transitions are handled via UnityEngine.SceneManagement.SceneManager.LoadScene():

```
// GameManager.cs — on Game Over
SceneManager.LoadScene("EndGame");

// MainMenuController.cs — on Play button
SceneManager.LoadScene("Gameplay");
```

3.2. In-Game HUD

The Gameplay scene features a Canvas-based HUD with the following elements:

· Score Text (TextMeshProUGUI) — top-left, updated in real-time via GameManager.UpdateUI().

· Level Text (TextMeshProUGUI) — displayed below score, updates on every level-up event.

· High Score Text (TextMeshProUGUI) — reads from PlayerPrefs and displays the all-time best run.

· Boss Health Bar (UI Slider) — hidden by default; activates when a Boss spawns. Labeled "ALIEN DREADNOUGHT".

```
// GameManager.cs — UpdateUI()
if (scoreText != null)     scoreText.text     = "Score: " + score;
if (levelText != null)     levelText.text     = "Level: " + level;
if (highScoreText != null) highScoreText.text = "Best: "  + highScore;
```

IMAGE 2: In-game screenshot showing the HUD and Boss Health Bar slider during a boss encounter.

4. Core Mechanics & Physics Scripting

4.1. Movement and Transform Logic

Player Movement

PlayerController.cs reads raw axis input each frame and applies it to Rigidbody2D.linearVelocity. A Mathf.Clamp boundary system prevents the ship from leaving the visible screen:

```
void Update()
{
    float moveX = Input.GetAxisRaw("Horizontal");
    float moveY = Input.GetAxisRaw("Vertical");
    rb.linearVelocity = new Vector2(moveX, moveY).normalized * moveSpeed;

    Vector3 pos = transform.position;
    pos.x = Mathf.Clamp(pos.x, minX, maxX);   // -8 to +8
    pos.y = Mathf.Clamp(pos.y, minY, maxY);   // -4 to +4
    transform.position = pos;
}
```

Falling Entities (Asteroids & Stars)

Asteroid.cs and Star.cs use transform.Translate() in Update() to move strictly downward in World Space, decoupled from the object's own rotation:

```
// Asteroid.cs
transform.Translate(Vector3.down * fallSpeed * Time.deltaTime, Space.World);
transform.Rotate(0, 0, rotateSpeed * Time.deltaTime); // independent spin
```

Boss Movement

Boss.cs directly sets transform.position each frame using a sinusoidal horizontal sweep:

```
transform.position = new Vector3(
    Mathf.Sin(Time.time * horizontalSpeed) * horizontalRange,
    3.5f, 0f
);
```

4.2. Physics and Collision Detection

All GameObjects use 2D Trigger Colliders (IsTrigger = true) with Rigidbody2D components, enabling the OnTriggerEnter2D(Collider2D other) callback without physical forces.

Combat — Laser vs. Asteroid

```
// Asteroid.cs
if (other.CompareTag("Laser"))
{
    GameManager.Instance.AddScore(scoreWhenDestroyed); // +5 pts
    Destroy(other.gameObject);
    Destroy(gameObject);
}
```

Scoring — Player vs. Star

```
// Star.cs
if (other.CompareTag("Player"))
    GameManager.Instance.AddScore(scoreValue); // +10 pts
```

Instant Death — Asteroid / Boss Bullet vs. Player

Per the final game design, any single collision with a hazard triggers immediate Game Over — no HP system:

```
// Asteroid.cs
if (other.CompareTag("Player"))
{
    PlayCrashFeedback();
    GameManager.Instance.TriggerGameOver();
    Destroy(gameObject);
}

// EnemyBullet.cs
if (other.CompareTag("Player"))
{
    GameManager.Instance.TriggerGameOver();
    Destroy(gameObject);
}
```

Boss Combat — Laser vs. Boss

Boss.cs implements a 50 ms invincibility window between hits to prevent multi-bullet damage in the same physics step. After 30 hits, the boss is destroyed:

```
private void OnTriggerEnter2D(Collider2D other)
{
    if (!other.CompareTag("Laser")) return;
    if (Time.time < invincibleUntil) return;

    invincibleUntil = Time.time + HIT_COOLDOWN;
    Destroy(other.gameObject);
    health--;
    GameManager.Instance.UpdateBossHealthUI(health);
    if (health <= 0) { GameManager.Instance.NotifyBossDestroyed();
```

```
Destroy(gameObject); }
}
```

4.3. Score-Based Level Progression

Levels advance dynamically whenever the player's score crosses a 100-point threshold:

```
// GameManager.cs — AddScore()
int newLevel = (scorePerLevel > 0) ? (score / scorePerLevel) + 1 : 1;
if (newLevel > level)
{
    for (int i = 0; i < newLevel - level; i++) { level++; OnLevelUp(); }
}
```

Level Range	Effect
Every level	Spawn delay reduced by 0.15 s
Level 5, 10, 15…	Boss spawns
Level 6, 11, 16, 21…	MILESTONE: fall speed +0.8, extra density burst

4.4. Dynamic Memory Optimization (Garbage Collection)

To prevent unbounded memory growth from off-screen objects, every spawned entity schedules its own Destroy() in Start():

```
// Asteroid.cs
void Start() { Destroy(gameObject, 10f); } // auto-clear after 10 s

// EnemyBullet.cs
void Start() { Destroy(gameObject, 5f); }  // faster for bullets

// Star.cs
void Start() { Destroy(gameObject, 12f); }
```

Additionally, a static ActiveCount counter on Asteroid, Star, and PowerUp classes prevents the spawner from over-populating the scene beyond configured maximums (maxActiveAsteroids = 20, maxActiveStars = 5).

5. Creative Implementations (Polish & Game Feel)

5.1. Parallax Scrolling Background System (Reserved for future use)

Three distinct background layers (Moving, Mid, Near) were built using prefabs with BackgroundScroller.cs. Each layer scrolls at a different speed to create a convincing depth-of-field parallax illusion.

BackgroundManager.cs switches the active background set as levels advance, reinforcing the sense of progression through deeper space:

```
// BackgroundManager.cs
private void UpdateBackground()
{
    int idx = Mathf.Clamp((level - 1) / levelsPerBackground, 0,
backgrounds.Length - 1);
    // activates Moving → Mid → Near prefab set per tier
}
```

5.2. Audio Feedback System (SFX & BGM)

· Background Music (BGM): An AudioSource on the Main Camera loops a Sci-Fi track throughout gameplay. It is programmatically stopped on Game Over:

```
// GameManager.cs — GameOver()
AudioSource cameraAudio = Camera.main.GetComponent<AudioSource>();
if (cameraAudio != null) cameraAudio.Stop();
```

· Sound Effects (SFX): AudioSource.PlayClipAtPoint() fires uncompressed .wav clips on key events: crashSound (asteroid destroyed / player collision) and star pickup sound.

IMAGE 3: Inspector screenshot of the Asteroid prefab showing the Audio Clip field populated with the crash SFX.

5.3. Kinesthetic Feedback — Camera Shake

CameraShake.cs implements a Coroutine-driven screen shake effect triggered on all violent collisions. A randomised Vector3 offset (Random.Range) is applied to the camera's local position for 0.5 seconds with 0.2 magnitude, then snaps back to origin:

```
// CameraShake.cs
public IEnumerator ShakeCoroutine(float duration, float magnitude)
{
    float elapsed = 0f;
    while (elapsed < duration)
    {
        transform.localPosition = Random.insideUnitSphere * magnitude;
        elapsed += Time.deltaTime;
        yield return null;
    }
    transform.localPosition = Vector3.zero;
}
```

5.4. Boss Encounter System

At every 5th level, GameManager.TrySpawnBoss() spawns the Alien Dreadnought boss. The boss:

· Patrols the top of the screen horizontally via a sine-wave path.

· Fires EnemyBullet projectiles downward at a set fireRate.

· Displays a dedicated health slider UI that depletes in real-time as the player shoots it.

· Awards +200 bonus points upon defeat via NotifyBossDestroyed().

· Asteroid spawning is paused while the boss is alive, focusing the challenge.

5.5. Data Persistence — PlayerPrefs

The player's final score is persisted across scene boundaries using Unity's PlayerPrefs:

```
// GameManager.cs — GameOver()
if (score > PlayerPrefs.GetInt("HighScore", 0))
    PlayerPrefs.SetInt("HighScore", score);

PlayerPrefs.SetInt("FinalScore", score);
PlayerPrefs.Save();
```

EndGameManager.cs reads FinalScore on load to display the result and compute a rank title (e.g., Space Admiral, Elite Pilot) based on score thresholds.

6. Conclusion

The Space Explorer project successfully delivers a complete, polished 2D arcade experience that satisfies all lab requirements while incorporating significant creative enhancements.

Key technical achievements:

Area	Implementation
Scene Management	3-scene flow with SceneManager
Physics	2D Trigger Colliders + OnTriggerEnter2D
Spawning	Coroutine-driven with density caps and difficulty scaling
Progression	Score-based level system (100 pts/level)
Boss System	Per-milestone spawn, health UI, reward on defeat
Audio	BGM loop + per-event SFX via PlayClipAtPoint

Visual Feedback	Parallax background, camera shake, shield blink
Memory	Auto-Destroy() + static ActiveCount limiters
Data	PlayerPrefs for cross-scene score persistence

The project demonstrates proficiency in Unity's core systems and reflects a well-structured, maintainable codebase suitable for continued feature development.

End of Lab 1 Report — Space Explorer | QE190017 — Nguyễn Công Nhất